Deferrable Server for Aperiodic Task Handling in Fixed-Priority Real-Time Systems

MD Rafiul Islam
Student ID: 1232714
Real-Time Systems Seminar
Hamm-Lippstadt University of Applied Sciences

***Abstract*—Real-time systems often execute hard periodic tasks together with aperiodic requests whose arrival times are not known in advance. Serving such requests only in background time protects the periodic workload, but it usually gives poor response times. A Deferrable Server addresses this problem by reserving a fixed amount of processor capacity for aperiodic work and preserving unused capacity until the end of the server period. This paper explains the motivation, mathematical model, algorithmic rule, timing impact, runtime overhead, and implementation aspects of the Deferrable Server. It also discusses the main limitation of the method: deferred capacity can create bursty interference near period boundaries. A small UPPAAL model is used to demonstrate the central behavior with a 10 ms period, a 2 ms budget, and an aperiodic job arriving at 6 ms. The model is deliberately small, because its purpose is to explain the main idea clearly rather than to replace a full schedulability proof.**

***Index Terms*—Deferrable Server, fixed-priority scheduling, aperiodic tasks, real-time systems, UPPAAL, schedulability.**

I. Introduction

The correctness of a real-time system depends not only on the logical value of a result, but also on the time at which the result is produced. A control output that arrives after its deadline may be useless or unsafe. For this reason, real-time scheduling focuses on predictability rather than only processor speed [1].

Many embedded applications contain both periodic and aperiodic activities. Periodic tasks are released at known intervals. Examples include sensor sampling, control loops, actuator updates, and cyclic monitoring. Aperiodic jobs are different because they arrive irregularly. Examples include operator commands, diagnostic messages, non-critical alarms, and network packets.

This creates a scheduling problem. If aperiodic jobs are executed only when the processor is idle, hard periodic tasks remain protected, but aperiodic response time may become poor. If aperiodic jobs are executed immediately without control, they may disturb the timing guarantees of hard periodic tasks. Fixed-priority servers solve this conflict by assigning a controlled amount of processor time to aperiodic work [1].

The Deferrable Server is one of the classical fixed-priority servers. It improves aperiodic response time compared with the Polling Server by preserving unused capacity until the end of the server period. This makes the method simple and responsive, but not without cost. The preserved capacity can create extra interference near period boundaries.

II. Background

A. Periodic and Aperiodic Tasks

A periodic task τ_i is commonly described by a worst-case execution time C_i and a period T_i. Its utilization is

$$U_i = \frac{C_i}{T_i}. \tag{1}$$

For a set of periodic tasks, the total periodic utilization is

$$U_p = \sum_i \frac{C_i}{T_i}. \tag{2}$$

Under Rate Monotonic scheduling, shorter-period tasks receive higher priorities. Since the priorities are assigned before runtime, Rate Monotonic belongs to fixed-priority scheduling [2].

Aperiodic jobs are not released at regular intervals. They may arrive early, late, in bursts, or not at all. In many practical systems, such jobs are soft or firm rather than hard. Late service is undesirable, but it is usually not as dangerous as missing a hard control deadline.

B. Why Servers Are Needed

A server is a scheduled entity used to execute aperiodic jobs. It has a period T_s and a capacity C_s. Its utilization is

$$U_s = \frac{C_s}{T_s}. \tag{3}$$

The purpose of this budget is to limit how much processor time aperiodic work can consume. In this way, the system can react to irregular events while still keeping the periodic workload analyzable.

A Polling Server checks for aperiodic jobs at its activation time. If no job is waiting, the server capacity is lost. This is simple, but it may cause unnecessary waiting. If a job arrives shortly after the polling instant, it may need to wait until the next server period. The Deferrable Server changes exactly this behavior.

III. Deferrable Server Mechanism

The Deferrable Server has a capacity C_s, a period T_s, and a fixed priority level. At the beginning of every server period, its capacity is replenished to C_s. If an aperiodic job is pending and the remaining capacity is greater than zero, the server

executes that job. During execution, the remaining capacity is reduced. If no aperiodic job is pending, the capacity is not lost immediately. It is kept available until the end of the current server period.

The rule can be described in five steps. First, at the start of each server period, the remaining capacity is set to C_s. Second, if an aperiodic job is pending and the remaining capacity is positive, the server executes it at the server priority. Third, during this execution the capacity is decreased. Fourth, if no job is pending, the remaining capacity is preserved. Finally, at the next server period, the capacity is replenished again.

This behavior is the main difference from a Polling Server. The processor is not used unnecessarily when no aperiodic job exists, but the right to serve future aperiodic work inside the same period is preserved.

For example, let

$$T_s = 10 \text{ ms}, \qquad C_s = 2 \text{ ms}. \tag{4}$$

If no aperiodic job is present at $t = 0$, a Polling Server would discard its capacity. A Deferrable Server keeps it. If a job arrives at $t = 6$ ms and requires 2 ms of execution, the Deferrable Server can execute it immediately from $t = 6$ ms to $t = 8$ ms. This is the main response-time advantage of DS.

IV. Formal Description of the Algorithm

A compact way to describe the server is to track the remaining budget as a state variable. Let $cap(t)$ denote the remaining server capacity at time t. At each server boundary kT_s, the capacity is replenished:

$$cap(kT_s) = C_s. \tag{5}$$

During aperiodic execution, the budget decreases with the amount of service already provided. If no aperiodic job is pending, the value of $cap(t)$ is kept unchanged until the end of the current server period. If the current period ends and the capacity has not been used, the old value is discarded and the next period begins with a fresh budget.

The scheduling decision can be read as a simple priority rule. If aperiodic work is waiting and $cap(t) > 0$, the server is ready at its assigned priority. If no such work exists, the server does not execute, but the budget remains stored. This is why the method is called deferrable: the service opportunity is deferred to a later arrival inside the same period.

For hard periodic tasks, the server must still be included in the workload analysis. The server behaves like an additional high-priority activity, but its interference is not identical to a normal periodic task because the execution can be shifted inside the period. This shifted execution is the reason why DS needs more careful analysis than a simple periodic budget reservation.

V. Schedulability and Main Limitation

The strength of the Deferrable Server also creates its main weakness. Because unused capacity can be deferred, the server may execute near the end of one period and then again immediately after the next replenishment. A typical situation is execution in

$$[8, 10) \text{ ms} \tag{6}$$

using saved capacity, followed by execution in

$$[10, 12) \text{ ms} \tag{7}$$

using newly replenished capacity. This back-to-back behavior can create stronger local interference than a normal periodic task with the same C_s and T_s.

Therefore, DS cannot always be analyzed as an ordinary periodic task. A normal high-priority periodic task executes when it is released. A Deferrable Server may preserve its capacity and execute later, depending on the arrival of an aperiodic job. This changes the interference pattern seen by lower-priority periodic tasks.

The practical consequence is that DS improves aperiodic response time, but it may reduce the schedulability margin of periodic tasks. The choice of C_s and T_s must therefore be made carefully. A larger capacity improves aperiodic service but increases interference. A smaller capacity protects periodic tasks better but may increase aperiodic waiting time.

VI. Runtime and Implementation Overhead

The runtime mechanism of DS is not complicated. The scheduler must maintain the current server capacity, the current server period, and the queue of aperiodic jobs. At a period boundary, the capacity is reset to C_s. When the server executes an aperiodic job, the capacity is decreased. If the capacity reaches zero, the server cannot execute more aperiodic work until the next period.

The algorithmic overhead is mainly budget accounting and queue management. With a simple first-in first-out queue, inserting and removing aperiodic jobs can be implemented efficiently. However, in a real implementation, overhead also comes from timer handling, context switches, interrupt latency, and the accuracy of budget measurement. These details are often abstracted in theory, but they matter in embedded systems.

VII. Comparison with Related Servers

Different fixed-priority servers handle unused capacity in different ways. Background service is the simplest method, because aperiodic jobs run only when the processor is idle. This is safe for periodic tasks, but response time can be poor. A Polling Server introduces a fixed budget, but it loses the budget if no request is waiting at the polling instant. A Deferrable Server improves this by preserving unused budget until the end of the period. The cost is that the preserved budget can later appear as a burst of interference.

Compared with DS, the Sporadic Server gives better control over replenishment and interference, but it requires more bookkeeping because replenishment times must be tracked carefully [4]. Priority Exchange also tries to reuse unused capacity more carefully, but it is harder to explain and implement than DS. Therefore, DS is a useful middle point: it is more responsive

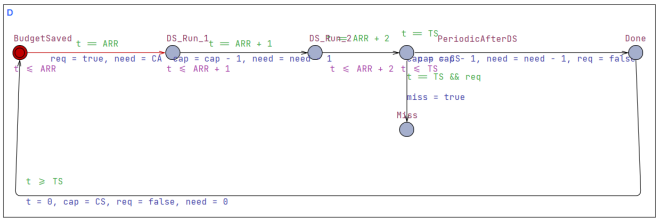

Fig. 1. Full UPPAAL model of the Deferrable Server demonstration.

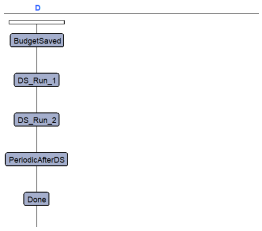

Fig. 2. Simplified state sequence used to explain the UPPAAL trace.

than a Polling Server and simpler than Sporadic Server or Priority Exchange.

VIII. UPPAAL Demonstration

The UPPAAL model was built to make the core DS behavior visible. It is not a complete model of a real-time operating system. Instead, it focuses on one question: if an aperiodic job arrives after the beginning of the server period, is the saved DS capacity still available?

The model uses $T_s = 10$ ms, $C_s = 2$ ms, aperiodic arrival time $ARR = 6$ ms, and aperiodic execution time $C_A = 2$ ms. The model contains a clock t, a capacity variable cap, a remaining execution variable $need$, a request flag req, and a miss flag $miss$.

The full UPPAAL model is shown in Fig. 1. It is placed first because it shows the actual timed automaton used in the experiment. The model starts in *BudgetSaved*. At $t = 6$ ms, the aperiodic request arrives. The transition sets the request flag to true and sets the remaining work to C_A. The important point is that the capacity variable is still equal to C_s at this moment. The model then moves through *DS Run 1* and *DS Run 2*, which represent the two milliseconds of aperiodic execution. After that, the request is finished and the system waits until the end of the period.

For presentation, the same behavior can be simplified as the state sequence shown in Fig. 2. This simplified version is easier to explain to an audience because it hides the detailed guards and updates and only shows the main execution path.

The transition from *Done* back to *BudgetSaved* represents the next server period. This transition was added only to avoid an artificial final deadlock in the simulator. It does not change the meaning of the first server period; it only allows the model to continue cyclically.

IX. Simulation and Verification Results

The concrete simulator follows one execution trace. In the trace, the model stays in *BudgetSaved* from 0 ms to 6 ms. No aperiodic job exists during this time, but the capacity is not discarded. At 6 ms, the job arrives and DS immediately starts execution because the budget is still available.

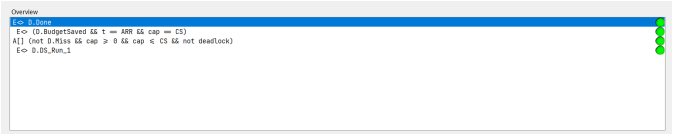

Fig. 3. UPPAAL verifier results for the Deferrable Server model. The green indicators show that the selected reachability and safety properties are satisfied.

The timeline is simple. At 0 ms, the server period starts with 2 ms of capacity. From 0 ms to 6 ms, the model remains in *BudgetSaved*; this is the budget-preservation part. At 6 ms, the aperiodic job arrives and the model enters *DS Run 1*. At 7 ms, one millisecond has been served and the model enters *DS Run 2*. At 8 ms, the job is complete and the model enters *PeriodicAfterDS*. At 10 ms, the period is complete and the model reaches *Done*.

The verifier was used to check the intended behavior. The first property checks that a successful state is reachable. The second checks that the model reaches the server execution state. The third property checks the main DS idea: at the arrival time, the model can still be in *BudgetSaved* with full capacity. The final safety property checks that the error state is avoided, capacity remains valid, and the model has no deadlock.

As shown in Fig. 3, all selected verification properties are satisfied. The model can reach the successful *Done* state, the server execution state is reachable, the saved capacity is available at the aperiodic arrival time, and the model avoids the miss state and deadlock. These checks support the intended explanation of the Deferrable Server behavior.

The most important verification result is the budget-preservation property. It confirms that at $t = 6$ ms, when the aperiodic job arrives, the server still has its full capacity. This is exactly the difference between the Deferrable Server and the Polling Server in the selected example.

These results do not prove that every possible DS implementation is schedulable. They prove that this small model behaves as intended. For the seminar, this is useful because it connects the abstract server rule with a concrete timed automaton.

X. Parameter Choice and Engineering Tradeoff

The parameters of the Deferrable Server must be chosen according to the expected aperiodic workload and the available processor margin. A small value of C_s gives strong protection to periodic tasks, but it may not be enough to serve an aperiodic job quickly. A large value of C_s improves aperiodic response time, but it increases the interference caused by the server. Similarly, a short period T_s replenishes capacity more often, while a long period reduces replenishment frequency but may increase waiting time after the budget has been consumed.

A practical design procedure is therefore iterative. First, the hard periodic task set is analyzed without a server. Second, a tentative server utilization $U_s = C_s/T_s$ is selected from the remaining processor margin. Third, the periodic task set

is rechecked including the server interference. Finally, the response time of representative aperiodic jobs is evaluated. If periodic tasks become unsafe, the server budget must be reduced or the period must be increased. If aperiodic response time is too poor, the budget must be increased only if the periodic workload still remains schedulable.

This tradeoff is the reason why DS is not simply a performance optimization. It is a scheduling mechanism with a clear cost. The designer buys faster aperiodic response by reserving processor capacity and by accepting a more complicated interference pattern. In a safety-critical system, this cost must be documented and justified.

XI. Application Example

A realistic use case for DS is a control device that runs hard periodic control loops and sometimes receives non-critical diagnostic requests. The periodic tasks may sample sensors and update actuators every few milliseconds. These tasks must remain predictable. At the same time, the system should respond quickly when an operator asks for status information or when a diagnostic event is received.

In such a case, DS can reserve a small amount of CPU time for the diagnostic channel. If no diagnostic request arrives, the reserved capacity is kept for later in the same server period. If a short request arrives, it can be served quickly. If many diagnostic requests arrive continuously, the capacity limit prevents them from consuming unlimited CPU time. Therefore, DS is suitable for short and irregular aperiodic jobs, but not for heavy aperiodic workloads with hard deadlines.

XII. When DS Is Suitable and Unsuitable

The Deferrable Server is most suitable when the aperiodic jobs are short, irregular, and useful mainly for responsiveness. Examples are user commands, status requests, lightweight diagnostics, and event messages that should be handled quickly but do not by themselves define the safety of the system. In such cases, the server budget can be small, and the preserved capacity can noticeably improve response time.

The method is less suitable when aperiodic jobs are long, frequent, or safety-critical. Long jobs may exhaust the budget quickly and then wait for the next replenishment. Frequent jobs may keep the server busy in many periods, increasing interference on lower-priority tasks. Safety-critical jobs may need their own hard real-time analysis rather than being treated as soft aperiodic work. In those cases, a Sporadic Server, a reservation-based method, or a separate hard periodic task may be a better design choice.

The model in this paper should therefore be interpreted carefully. It demonstrates the best-known positive behavior of DS: the saved budget is immediately available when the request arrives. It does not demonstrate overload behavior, multiple queued aperiodic jobs, different server priorities, shared resources, or preemption overhead. These are important topics for a complete implementation, but they would make the small UPPAAL model harder to explain within a short seminar.

XIII. Critical Discussion

The Deferrable Server is useful because it is simple and responsive. It is easier to implement than some more advanced server mechanisms, and its behavior can be explained with a short timing example. These properties make it attractive for small embedded systems and for teaching fixed-priority server concepts.

However, DS should not be treated as a perfect solution. Its preserved capacity can cause back-to-back execution and stronger interference on lower-priority tasks. This means that a periodic task set may need a more conservative schedulability test. The method is also sensitive to the choice of C_s and T_s. A large capacity improves aperiodic response time but increases interference. A small capacity protects periodic tasks better but may make aperiodic jobs wait.

The UPPAAL model also has limitations. It abstracts away scheduler overhead, context-switch time, interrupt interference, and real queue behavior. It contains only one aperiodic job and a simple period repetition. Therefore, the model should be presented as a demonstration of the main feature, not as a complete proof of DS schedulability.

XIV. Conclusion

The Deferrable Server improves aperiodic responsiveness in fixed-priority real-time systems by preserving unused server capacity until the end of the server period. This makes it more responsive than a Polling Server, especially when an aperiodic job arrives after the server activation time. At the same time, the deferred capacity can create bursty interference near period boundaries, so careful schedulability analysis is required.

The UPPAAL experiment supports the central explanation. With a 10 ms server period, a 2 ms budget, and an aperiodic job arriving at 6 ms, the model shows that the budget remains available and the job can be served immediately from 6 ms to 8 ms. The verifier confirms that the successful state is reachable, the server starts execution, the capacity is preserved at the arrival time, and the error state is avoided. Overall, DS is a practical compromise: simple, responsive, and useful for short aperiodic jobs, but not free from timing risks.

References

[1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.

[2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[3] J. P. Lehoczky, L. Sha, and J. K. Strosnider, "Enhanced aperiodic responsiveness in hard real-time environments," in *Proc. IEEE Real-Time Systems Symposium*, 1987, pp. 261–270.

[4] B. Sprunt, L. Sha, and J. P. Lehoczky, "Aperiodic task scheduling for hard-real-time systems," *Real-Time Systems*, vol. 1, no. 1, pp. 27–60, 1989.

[5] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[6] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems*. Berlin, Germany: Springer, 2004, pp. 200–236.

[7] L. Abeni and G. Buttazzo, "Integrating multimedia applications in hard real-time systems," in *Proc. IEEE Real-Time Systems Symposium*, 1998, pp. 4–13.